To ensure data consistency and efficient linking between entities, multiple relationships were established:

1.One-to-One Relationship (User - Student)

Each user can have a corresponding student profile.

Implemented via :

`hasOne(models.student, { foreignKey: 'student_id' })`.

2.One-to-Many Relationship (User - Phone Numbers)

A user can have multiple registered phone numbers.

Implemented via :

`hasMany(models.phone_number, { foreignKey: 'user_id' })`.

3.One-to-One Relationship (User - Doctor)

Each user can have a corresponding doctor profile.

Implemented via :

`hasOne(models.doctor, { foreignKey: 'doctor_id' })`.

4.One-to-Many Relationship (User - Notifications)

A user can send multiple notifications.

Implemented via :

`hasMany(models.notification, { foreignKey: 'sender_id' })`.

5. One-to-Many Relationship (User - Books)

A user can add multiple books.

Implemented via :

`hasMany(models.book, { foreignKey: 'added_by' })`.

6. Many-to-One Relationship (User - Section)

Each user belongs to a specific section.

Implemented via :

`belongsTo(models.section, { foreignKey: 'user_section_id' })`.

7. Many-to-One Relationship (User - Role)

Each user is assigned a specific role in the system.

Implemented via :

`belongsTo(models.role, { foreignKey: 'roleId' })`.

Database Security Measures

To protect sensitive data, various security measures were implemented:

- ★ Default Scope and Scoes By default field like password is excluded.
- ★ Password Hashing using `bcrypt` to encrypt user passwords.
- ★ Access Control through `roleId` to define different user permissions.
- ★ Token-based Authentication using `JWT` for secure session management.

8. Subject Table

The subject table represents academic subjects or courses offered by an institution. It has several relationships with other tables, making it a central part of the database structure.

Database Schema Design

This table stores information about subjects offered in the academic program.

The subject table includes the following columns

Column Name	Data Type	Description
subject_id	STRING(10)	(PK)A unique identifier for each subject, such as "MATH101".
subject_name	JSON	The name of the subject, potentially stored in multiple languages.
number_of_units	TINYINT	The number of academic units or credits assigned to the subject.
subject_description	JSON	A description of the subject, stored in multiple languages if needed.

Database Relationships

- One-to-Many Relationships:

study_plan_elment table: Each subject can be part of many study plan elements.

grade table: Each subject can have multiple grade entries.

prerequisite table: A subject can have multiple prerequisites.

exam table: A subject can have multiple associated exams.

book table: A subject can have multiple books associated with it.

assignment table: A subject can have multiple assignments.

- Many-to-Many Relationship:

doctor table: Each subject can have multiple doctors (teachers) associated with it through a junction table subject_teacher.

9. Student Table

The student table holds data about students, including their user details, academic information, enrollment year, and the courses they're registered for.

Database Schema Design

The student table stores personal and academic data about the students.

The student table includes the following columns:

Column Name	Data Type	Description
student_id	INTEGER	(PK) and (FK) The unique identifier for each student, linked to the users table.
study_plan_id	INTEGER	The ID of the student's study plan, linked to the study_plans table.
student_level_id	INTEGER	The student's level ID, linked to the levels table.
enrollment_year	DATEONLY	Student enrollment year.
student_system	JSON	System data related to the student, such as (General,FreeSeat,Paid), potentially stored in multiple languages.

repeat_years_count	INTEGER	The number of years the student has repeated.

Database Relationships

- One-to-One Relationships:

user table: Each student is linked to a user, as the student_id in the student table is a foreign key referring to the user_id in the user table.

- One-to-Many Relationships:

study_plan table: Each student can have one study plan. The study_plan_id is a foreign key in the student table referring to the study_plans table.

grade table: Each student can have multiple grades, associated by the student_id foreign key in the grade table.

student_fee table: A student can have multiple fee records, with the student_id in the student_fee table.

student_assignment table: Each student can have multiple assignments. The student_id is used to reference the student in the student_assignment table.

- Many-to-Many Relationships:

assignment table: Students are linked to assignments via a many-to-many relationship through the student_assignment junction table.

Default Scope and Exclusion

The default scope excludes certain attributes (like study_plan_id and student_level_id) from being returned in the response by default. However, it still includes related models such as study_plan and level.

```
101     defaultScope: {
102       attributes: { exclude: ['study_plan_id', 'student_level_id'] },
103       include: [
104         {
105           association: 'study_plan',
106
107         },
108         {
109           association: 'level',
110
111         }
112       ],
113     },
114
```

Security and Integrity

Cascading Deletes and Updates:

The foreign keys in the student table are set to CASCADE for updates and deletes, ensuring that changes made to associated records (like users or study plans) are reflected in the student data.

```
student_id: {
  type: DataTypes.INTEGER,
  primaryKey: true,
  references: {
    model: 'users',
    key: 'user_id',
  },
  onDelete: 'CASCADE',//if a user is delete the student associated with him will be deleted
  onUpdate: 'CASCADE',//if a user is update the student associated with him will be updated
},
```

Null Handling for Study Plan:

If a study plan is deleted, the study_plan_id for the student is set to NULL, ensuring data integrity without losing student records.

```
study_plan_id: {
  type: DataTypes.INTEGER,
  allowNull: true,
  references: {
    model: 'study_plans',
    key: 'study_plan_id',
  },
  onDelete: 'SET NULL',//if a study_plan is delete the student associated with him will be set null in
  onUpdate: 'CASCADE',//if a study_plan is update the student associated with him will be updated
},
```

Methods and Extra Functionality

getFullData Method:

This method merges the student's data with the related user data. If a student has an associated user, it combines the student and user data and returns it as a single object, excluding unnecessary fields like user.

```
118
119  student.prototype.getFullData = function () {
120    const student = this.toJSON();
121    if (this.user) {
122      delete student.user;
123      return { ...this.user.toJSON(), ...student };
124    }
125    return student;
126  };
```

10. Phone number Table

The phone number table stores phone numbers for users, allowing multiple phone numbers to be associated with a single user.

Database Schema Design

The phone number table stores phone numbers, with each number linked to a specific user.

The phone number table includes the following columns

Column Name	Data Type	Description
user_id	INTEGER	(FK)The ID of the user to whom the phone number belongs,Linked to users.
phone_number	STRING(25)	The phone number associated with the user.

Database Relationship

One-to-Many Relationship:

user table: The phone_number table has a foreign key user_id that references the user_id in the user table. This allows a user to have multiple phone numbers.

Cascading Updates and Deletes

The user_id foreign key in the phone_number table is set to CASCADE for both onDelete and onUpdate. This ensures that if a user is deleted or updated, the related phone numbers are also deleted or updated accordingly.

11. doctor Table

The doctor table stores information about the doctors, each of whom is associated with a user record and can have multiple subjects, assignments, and study plan elements.

Database Schema Design

The doctor table stores personal and professional information about doctors and teaching staff, with a reference to the user table for user-related details.

The doctor table includes the following columns

Column Name	Data Type	Description
doctor_id	INTEGER	(PK) and (FK) The ID of the doctor, referencing the user_id from the users table.
academic_degree	JSON	The academic degree(s) of the doctor. potentially stored in multiple languages.
administrative_position	JSON	The administrative position of the doctor (default is 'None'). potentially stored in multiple languages.

Database Relationship

- One-to-One Relationship:

user table: The doctor table has a foreign key doctor_id that references user_id in the user table, meaning that each doctor corresponds to a single user record.

- One-to-Many Relationships:

study_plan_elment: A doctor can have many study plan elements assigned to them, linked via doctor_id.

assignment: A doctor can be assigned many assignments, with each assignment associated with a specific doctor via doctor_id.

- Many-to-Many Relationship:

subject table: A doctor can teach many subjects and vice versa. This relationship is managed through a junction table called subject_teacher, with doctor_id linking doctors and subjects

12. Subject teacher Table

The subject teacher table stores the relationship between doctors and the subjects they teach. This table connects the subjects table with the doctors table.

Database Schema Design

The subject teacher table is a junction table used to establish a many-to-many relationship between subjects and doctors.

The subject teacher table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the relationship.
subject_id	STRING(10)	The ID of the subject being taught, referencing subject_id in the subjects table.
doctor_id	INTEGER	The ID of the doctor teaching the subject, referencing doctor_id in the doctors table.

Foreign Keys and Relationships

Foreign Keys :

- subject_id references the subject_id in the subjects table.
- doctor_id references the doctor_id in the doctors table.

Many-to-Many Relationship

This table serves as a junction between the subject and doctors, establishing a many-to-many relationship. A subject can have multiple doctors, and a permission can be assigned to multiple subjects

13. Grade Table

The grade table stores the grades of students across different subjects, with information about terms, sections, and levels.

Database Schema Design

The grade table includes the following columns

Column Name	Data Type	Description
student_id	INTEGER	A unique identifier for the grade record (Primary Key).

student_id	INTEGER	The ID of the student who receives the grade, referencing student_id in the students table.
subject_id	STRING(10)	The ID of the subject being graded, referencing subject_id in the subjects table.
exam_grade	TINYINT	The grade received by the student in the exam (optional).
work_grade	TINYINT	The grade received by the student for coursework or assignments (optional).
term	ENUM	The term in which the grade was issued (either 'Term 1' or 'Term 2').
section_id	INTEGER	The ID of the section for which the grade is issued, referencing id in the sections table.
level_id	INTEGER	The academic level of the student, referencing id in the levels table.
year_of_issue	DATEONLY	The year in which the grade was issued.
is_absent	BOOLEAN	Indicates whether the student was absent (true if absent, false if not).
status	JSON	Indicates whether the student is a Freshman or a Repeater. potentially stored in multiple languages.

Foreign Key and Relationships

Foreign Keys:

- student_id references the student_id in the students table.
- subject_id references the subject_id in the subjects table.
- section_id references the id in the sections table.
- level_id references the id in the levels table.

Cascading Updates and Deletes

If a student , subject , section, or level is deleted or updated, the associated records in the grade table will also be deleted or updated due to the CASCADE rules on the foreign keys.

14. Study plan element Table

The study plan element table stores data about each element of the study plan, including subjects, doctors, sections, levels, and terms.

Database Schema Design

The study plan element table includes the following columns:

Column Name	Data Type	Description
study_plan_elment_id	INTEGER	A unique identifier for the study plan element (Primary Key).
study_plan_id	INTEGER	The ID of the study plan this element belongs to, referencing study_plan_id in the study_plans table.
subject_id	STRING(10)	The ID of the subject, referencing subject_id in the subjects table.
doctor_id	INTEGER	The ID of the doctor assigned to the subject, referencing doctor_id in the doctors table.
section_id	INTEGER	The ID of the section, referencing id in the sections table.
level_id	INTEGER	The academic level associated with the study plan element, referencing id in the levels table.
number_of_units	INTEGER	The number of units or credits associated with this study plan element.
term	ENUM	The term in which the study plan element is to be taught (either 'Term 1' or 'Term 2').

Foreign Key and Relationships

Foreign Keys:

- study_plan_id references the study_plan_id in the study_plans table.
- subject_id references the subject_id in the subjects table.
- doctor_id references the doctor_id in the doctors table.
- section_id references the id in the sections table.
- level_id references the id in the levels table.

Cascading Updates and Deletes:

If a study_plan, subject, doctor, section, or level is deleted or updated, the associated records in the study_plan_element table will be deleted or updated according to the CASCADE rules

15. Prerequisite Table

The prerequisite table represents the prerequisites for subjects in the study plan. It links subjects with study plan elements, indicating the necessary subject dependencies for each element.

Database Schema Design

The prerequisite table links the subject with study_plan_element to define subject prerequisites.

The prerequisite table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the prerequisite record (Primary Key).
subject_id	STRING(10)	The ID of the prerequisite subject, referencing subject_id in the subjects table.
study_plan_elment_id	INTEGER	The ID of the study plan element this prerequisite belongs to, referencing study_plan_elment_id in the study_plan_elements table.

Foreign Key and Relationships

Foreign Keys :

- subject_id references the subject_id in the subjects table.
- study_plan_elment_id references the study_plan_elment_id in the study_plan_elments table.

Cascading Updates and Deletes :

If a subject or study_plan_element is deleted or updated, the corresponding prerequisite records will be deleted or updated accordingly, based on the CASCADE rule.

Unique Constraints

The combination of study_plan_elment_id and subject_id must be unique to avoid duplicate records for prerequisites.

16. Student fee Table

The student fee table tracks the payment status of students' fees for each term and level. It associates with the student table and level table, managing details about the amount paid and the remaining amount.

Database Schema Design

The student fee table manages students' fee details, including total fees, paid amounts, and remaining fees for a specific term and level.

The student fee table includes the following columns:

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the fee record (Primary Key).
student_id	INTEGER	student's ID, referencing student_id in the students table.
level_fees_id	INTEGER	The level of the student, referencing id in the levels table.

term	ENUM('Term 1', 'Term 2')	The term for which the fee is applicable (either Term 1 or Term 2).
total_amount	DECIMAL(8, 2)	The total fee amount for the student in the given term and level.
amount_paid	DECIMAL(8, 2)	The amount already paid by the student for the term and level.
remaining_amount	DECIMAL(8, 2)	The remaining fee amount the student still owes.
payment_date	DATE	The date on which the fee was paid (nullable).
receipt_number	STRING	The receipt number associated with the payment.

Foreign Key and Relationships

- student_id references the student_id in the students table.
- level_fees_id references the id in the levels table.

Cascading Updates and Deletes

If a student or level record is deleted or updated, the related student_fee records will be deleted or updated accordingly based on the CASCADE rule

17. Lecture Table

The lecture table stores information about lectures, including the subject, doctor, section, level, and schedule details, along with the relationships for replacing lectures and tracking their status.

Database Schema Design

The lecture table tracks lectures assigned to specific subjects, doctors, sections, and levels, including schedule details like time, day, and room.

The lecture fee table includes the following columns:

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the lecture (Primary Key).

subject_id	STRING(10)	The subject associated with the lecture, referencing subject_id in the subjects table.
doctor_id	INTEGER	The doctor assigned to the lecture, referencing doctor_id in the doctors table.
lecture_section_id	INTEGER	The section in which the lecture takes place, referencing id in the sections table.
lecture_level_id	INTEGER	The level of the lecture, referencing id in the levels table.
term	ENUM('Term 1', 'Term 2')	The term for which the lecture is scheduled (either Term 1 or Term 2).
year	STRING(30)	The academic year during which the lecture is given.
lecture_time	TIME	The time at which the lecture takes place.
lecture_duration	INTEGER	The duration of the lecture in minutes.
lecture_day	ENUM	The day of the week on which the lecture occurs.
lecture_room	STRING(50)	The room where the lecture takes place (nullable).
lectureStatus	BOOLEAN	Specifies the lecture status(default is `true`).
isReplaced	BOOLEAN	Indicates whether the lecture has been replaced (default is `false`).
originalLecturId	INTEGER	The ID of the original lecture if this lecture replaces another, referencing id in the lectures table (nullable).

Foreign Key and Relationships

- subject_id references the subject_id in the subjects table.
- doctor_id references the doctor_id in the doctors table.
- lecture_section_id references the id in the sections table.
- lecture_level_id references the id in the levels table.

Self-referencing Relationships

- originalLecturId references the id in the lectures table.

Implemented via :

```
lecture.belongsTo(models.lecture, {
  foreignKey: 'originalLecturId',
  as: 'originalLecture',
});
```

Lecture Replacement System

- If a lecture is rescheduled or replaced, it refers back to original lecture :

  Lecture.originalLctureId→Lecture.id

- The alias **originalLecture** is used for the **originalLecture**.
- The alias **replacedLecture** hold all lectures replacing aspecific one.

Cascading Updates and Deletes:

- If a subject, doctor, section, or level record is deleted or updated, the related lecture record will be updated or deleted according to the CASCADE rule.
- Special Relationship for Replacing Lectures:
- originalLecturId links to another lecture if it replaces an existing one. The replaced lecture is tracked in the `replacedLecture` association.

Indexing

To improve performance and enforce uniqueness on certain fields, **unique index** is defined on the lecture table.

This index ensures that on two lectures can have the same **time, day, section, room,** and **replacement status** simultaneously.

This is crucial to prevent scheduling conflicts in the same section and room

Defined index in model :

```
  indexes: [
      {
        unique: true,
        fields: ['lecture_time', 'lecture_day',
'lecture_section_id', 'lecture_room','isReplaced'],
        name: 'unique_constraint_in_lecture',
      },
    ],
```

Defined index in migration :

```
    await queryInterface.addConstraint('lectures', {
      fields: ['lecture_time', 'lecture_day',
'lecture_section_id', 'lecture_room','isReplaced'],
      type: 'unique',
      name: 'unique_constraint_in_lecture',
    });
```

18. Exam Table

The exam table stores the details of exams including the subject, section, level, and specific details like exam date, time, and room. It defines the relationships between different entities, ensuring that exams are linked to specific subjects, sections, and levels.

Database Schema Design

The exam table is used to store details about each exam, including which subject, section, and level it is associated with, along with the exam's scheduling details.

The exam table includes the following columns

Column Name	Data Type	Description
exam_id	INTEGER	A unique identifier for the exam (Primary Key).
subject_id	STRING(10)	The subject associated with the exam, referencing subject_id in the subjects table.
exam_section_id	INTEGER	The section for the exam, referencing id in the sections table.
exam_level_id	INTEGER	The level for the exam, referencing id in the levels table.
exam_date	DATEONLY	The date when the exam is scheduled.
exam_time	TIME	The time when the exam starts.
exam_day	ENUM	The day of the week on which the exam occurs.
exam_room	STRING(50)	The room where the exam will take place.

Foreign Key and Relationships

- subject_id references subject_id in the subjects table.
- exam_section_id references id in the sections table.
- exam_level_id references id in the levels table.

Cascading Updates and Deletes

If a subject, section, or level record is deleted or updated, the related exam record will be updated or deleted according to the CASCADE rule.

Unique Constraints

The combination of exam_section_id, exam_level_id, exam_term, exam_year, exam_date, exam_time, and exam_day must be unique to prevent scheduling conflicts.

19. Notification Table

The notification table stores notifications sent to users. It defines the relationship between notifications and users, capturing details such as the message content, sender, and notification status.

Database Schema Design

The notification table stores information about notifications, including the sender, message, title, and notification type.

The notification table includes the following columns

Column Name	Data Type	Description
message_id	INTEGER	A unique identifier for the notification (Primary Key).
sender_id	INTEGER	The user who sent the notification, referencing user_id in the users table.
title	STRING(100)	The title of the notification.
message	TEXT	The content of the notification.
is_read	BOOLEAN	A flag indicating whether the notification has been read (default: `false`).
type	ENUM('System', 'Reminder', 'Alert')	The type of notification (default: `System`).

Foreign Key and Relationships

- sender_id references user_id in the users table.

Cascading Updates and Deletes

If a user record is deleted or updated, the related notification records will be updated or deleted according to the CASCADE rule.

20. Book Table

The book table is designed to manage and store information about books in a system. It defines relationships between books, users, subjects, levels, and sections. The table includes various fields to store book details such as the title, author, file information, and the user who added the book.

Database Schema Design

The book table includes the following columns:

Column Name	Data Type	Description

id	INTEGER	A unique identifier for the book (Primary Key).
section_id	INTEGER	The ID of the section the book belongs to (Foreign Key from sections).
level_id	INTEGER	The ID of the level the book is associated with (Foreign Key from levels).
title	STRING	The title of the book.
author	STRING	The author of the book (optional).
numberOfPages	INTEGER	The number of pages in the book (optional).
edition	STRING(50)	The edition of the book (optional).
category	ENUM	The category of the book (e.g., Book, Reference, etc.).
file_size	FLOAT	The file size of the book in bytes (optional).
file_path	TEXT	The path to the book file.
display_image	STRING	The image representing the book (optional).
added_by	INTEGER	The ID of the user who added the book (Foreign Key from users).
subject_id	STRING(10)	The ID of the subject the book is related to (Foreign Key from subjects).
original_name	STRING	The original name of the book file.

Foreign Key and Relationships

- `section_id` references `id` in the sections table.
- `level_id` references `id` in the levels table.
- `added_by` references `user_id` in the users table.
- `subject_id` references `subject_id` in the subjects table.

Many-to-Many Relationship:

The book table has many-to-many relationships with the level and section tables through the bookSectionLevel join table.

Cascading Updates and Deletes

If a referenced record in the users, subjects, levels, or sections table is deleted or updated, related records in the book table are updated or deleted accordingly.

21. Book Section Level Table

The bookSectionLevel table is a join table designed to represent the many-to-many relationships between books, sections, and levels. It helps link books to specific sections and levels, and ensures that each book can be associated with multiple sections and levels.

Database Schema Design

The Book Section Level table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
bookId	INTEGER	The ID of the book (Foreign Key from books).
sectionId	INTEGER	The ID of the section (Foreign Key from sections).
levelId	INTEGER	The ID of the level (Foreign Key from levels).

Foreign Key and Relationships

- `bookId` references the `id` in the books table.
- `sectionId` references the `id` in the sections table.
- `levelId` references the `id` in the levels table.

Unique Constraints

A unique constraint is applied on the combination of `bookId`, `sectionId`, and `levelId` to prevent duplicate associations between the same book, section, and level.

22. **Assignment**

The assignment table is used to store information about assignments. It includes relationships with multiple tables such as subjects, doctors, students, sections, and levels. Each assignment is linked to a specific subject, doctor, section, and level, with additional information like title, due day, and dates.

Database Schema Design

The assignment table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
subject_id	STRING(10)	The ID of the subject (Foreign Key from subjects).
doctor_id	INTEGER	The ID of the doctor (Foreign Key from doctors).
section_id	INTEGER	The ID of the section (Foreign Key from sections).
level_id	INTEGER	The ID of the level (Foreign Key from levels).
title	JSON	The title of the assignment (can be multi-lingual)
assignment_due_day	ENUM	The day of the week the assignment is due (e.g., Saturday).
assignment_date	DATE	The date the assignment was created
assignments_due_date	DATE	The deadline for the assignment.

Foreign Key and Relationships

- `subject_id` references the `subject_id` in the subjects table.
- `doctor_id` references the `doctor_id` in the doctors table.
- `section_id` references the `id` in the sections table.
- `level_id` references the `id` in the levels table.

Many-to-Many Relationship

students are related to assignments through the student_assignment table (many-to-many relationship).

23. Student Assignment Table

The student assignment table represents the relationship between students and assignments, specifically tracking which student is assigned to which assignment and the status of their submission. This table also tracks additional information, such as whether the assignment has been completed and any related files.

Database Schema Design

The Student Assignment table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
student_id	INTEGER	The ID of the student (Foreign Key from students).
assignment_id	INTEGER	The ID of the assignment (Foreign Key from assignments).
status	ENUM	The status of the assignment submission (Accepted, Rejected, etc.).
is_completed	BOOLEAN	Whether the student has completed the assignment.

Foreign Key and Relationships

- `student_id` references the `student_id` in the students table.
- `assignment_id` references the `id` in the assignments table.

Relationships:

- A student can have multiple assignments, and an assignment can be assigned to many students, so we use a many-to-one relationship between student_assignment and both the students and assignments tables.

```
student_assignment.belongsTo(models.assignment, {
  foreignKey: 'assignment_id',
});

student_assignment.belongsTo(models.student, {
  foreignKey: 'student_id',
});
```

- The student_assignment table also has a one-to-many relationship with the student_assignment_file table, allowing multiple files to be associated with each student's assignment.

```
student_assignment.hasMany(models.student_assignment_file,
{
    foreignKey: 'student_assignment_id',
 });
```

24. Assignment File Table

The assignment file table is designed to store files related to assignments. This table includes metadata about each file, such as its attachment, a hash for integrity validation, and the original file name. It is connected to the assignment table through a foreign key.

Database Schema Design

The assignment file table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
assignment_id	INTEGER	The ID of the assignment (Foreign Key from assignments).
attachment	TEXT	The file's attachment (usually in base64 format or file path).
attachment_hash	STRING(64)	A hash value of the file for validation purposes.
original_name	STRING	The original name of the file as uploaded by the user.

Foreign Key and Relationships

- `assignment_id` references the `id` in the assignments table.

Relationship

The assignment_file table is linked to the assignments table with a one-to-many relationship, meaning each assignment can have multiple files attached.

25. Student Assignment File Table

The student assignment file table is designed to store files related to student assignments. This table includes metadata about each file, such as its attachment, a hash for file integrity, and the original file name. It is associated with the student assignment table through a foreign key.

Database Schema Design

The student assignment file table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).

student_assignment_id	INTEGER	The ID of the student assignment (Foreign Key from student_assignments).
attachment	TEXT	The file's attachment (usually in base64 format or file path).
attachment_hash	STRING(64)	A hash value of the file for integrity validation.
original_name	STRING	The original name of the file as uploaded by the student.

Foreign Key and Relationships

- `student_assignment_id` references the `id` in the tudent_assignments table.

Relationship

The student_assignment_file table is linked to the student_assignments table with a one-to-many relationship, meaning each student assignment can have multiple files attached.

26. Refresh State Table

The refresh state table is used to store the state of a refresh operation, potentially for an application feature like session management or data filtering. It keeps track of the target of the refresh, the filter used (if any), and uses a unique identifier as the primary key.

Database Schema Design

The refresh state table includes the following columns

Column Name	Data Type	Description
id	STRING	A unique identifier for the record (Primary Key).
target	STRING	The target entity for which the refresh state is being tracked.

filter	JSON	A JSON field storing any filtering criteria for the refresh.

Foreign Key and Relationships

Foreign Key

- The refresh_state table does not include any foreign key relationships as it operates independently.

Relationships

There are no specific relationships defined for this table.